

Lecture 5: Operators in Real-Coded Genetic Algorithms

Selection, Crossover, Mutation, and Operator Behavior

Course: Evolutionary Algorithms

Instructor: Engr. Muhammad Farooq

Learning outcomes

By the end of this lecture, students should be able to:

- revise the main steps of a real-coded genetic algorithm

Learning outcomes

By the end of this lecture, students should be able to:

- revise the main steps of a real-coded genetic algorithm
- explain why selection still uses fitness values in RGA

Learning outcomes

By the end of this lecture, students should be able to:

- revise the main steps of a real-coded genetic algorithm
- explain why selection still uses fitness values in RGA
- distinguish $(\mu + \lambda)$ and (μ, λ) survival strategies

By the end of this lecture, students should be able to:

- revise the main steps of a real-coded genetic algorithm
- explain why selection still uses fitness values in RGA
- distinguish $(\mu + \lambda)$ and (μ, λ) survival strategies
- explain real-parameter crossover operators using simple examples

By the end of this lecture, students should be able to:

- revise the main steps of a real-coded genetic algorithm
- explain why selection still uses fitness values in RGA
- distinguish $(\mu + \lambda)$ and (μ, λ) survival strategies
- explain real-parameter crossover operators using simple examples
- describe BLX- α , SBX, fuzzy recombination, and UNDX conceptually

By the end of this lecture, students should be able to:

- revise the main steps of a real-coded genetic algorithm
- explain why selection still uses fitness values in RGA
- distinguish $(\mu + \lambda)$ and (μ, λ) survival strategies
- explain real-parameter crossover operators using simple examples
- describe BLX- α , SBX, fuzzy recombination, and UNDX conceptually
- explain random, non-uniform, normal, and polynomial mutation operators

By the end of this lecture, students should be able to:

- revise the main steps of a real-coded genetic algorithm
- explain why selection still uses fitness values in RGA
- distinguish $(\mu + \lambda)$ and (μ, λ) survival strategies
- explain real-parameter crossover operators using simple examples
- describe BLX- α , SBX, fuzzy recombination, and UNDX conceptually
- explain random, non-uniform, normal, and polynomial mutation operators
- connect operator parameters such as η_c and η_m with exploration and exploitation

Lecture roadmap

- 1 Quick recap of RGA

Lecture roadmap

- 1 Quick recap of RGA
- 2 Selection and survival strategies

Lecture roadmap

- 1 Quick recap of RGA
- 2 Selection and survival strategies
- 3 Real-parameter crossover operators

Lecture roadmap

- ➊ Quick recap of RGA
- ➋ Selection and survival strategies
- ➌ Real-parameter crossover operators
- ➍ Important properties of crossover operators

Lecture roadmap

- ➊ Quick recap of RGA
- ➋ Selection and survival strategies
- ➌ Real-parameter crossover operators
- ➍ Important properties of crossover operators
- ➎ Mutation operators for real-valued variables

Lecture roadmap

- ① Quick recap of RGA
- ② Selection and survival strategies
- ③ Real-parameter crossover operators
- ④ Important properties of crossover operators
- ⑤ Mutation operators for real-valued variables
- ⑥ Benchmark testing and final summary

Lecture roadmap

- 1 Quick recap of RGA
- 2 Selection and survival strategies
- 3 Real-parameter crossover operators
- 4 Important properties of crossover operators
- 5 Mutation operators for real-valued variables
- 6 Benchmark testing and final summary

The focus is not only on formulas. The focus is on **how each operator moves real-valued solutions in the search space.**

Recap of RGA

In the previous lecture, we studied the working principles of RGA.

- BGA represents variables using binary strings.

Recap of RGA

In the previous lecture, we studied the working principles of RGA.

- BGA represents variables using binary strings.
- RGA represents variables directly as real numbers.

Recap of RGA

In the previous lecture, we studied the working principles of RGA.

- BGA represents variables using binary strings.
- RGA represents variables directly as real numbers.
- This is suitable when the decision variables are continuous.

Recap of RGA

In the previous lecture, we studied the working principles of RGA.

- BGA represents variables using binary strings.
- RGA represents variables directly as real numbers.
- This is suitable when the decision variables are continuous.

Example chromosome:

$$X = [x_1, x_2, x_3] = [2.41, -0.82, 5.73]$$

Recap of RGA

In the previous lecture, we studied the working principles of RGA.

- BGA represents variables using binary strings.
- RGA represents variables directly as real numbers.
- This is suitable when the decision variables are continuous.

Example chromosome:

$$X = [x_1, x_2, x_3] = [2.41, -0.82, 5.73]$$

The general GA cycle remains the same: initialization, evaluation, selection, crossover, mutation, evaluation, and survival.

What changes from BGA to RGA?

Aspect	BGA	RGA
Representation	binary strings	real-valued vectors
Selection	based on fitness	based on fitness
Crossover	bit-level exchange	real-valued recombination
Mutation	bit flipping	real-valued perturbation
Decoding	required	usually not required

What changes from BGA to RGA?

Aspect	BGA	RGA
Representation	binary strings	real-valued vectors
Selection	based on fitness	based on fitness
Crossover	bit-level exchange	real-valued recombination
Mutation	bit flipping	real-valued perturbation
Decoding	required	usually not required

Selection logic remains almost unchanged because it only needs objective or fitness values. The main changes are in **crossover** and **mutation**.

Selection operators in RGA

- Selection operators need the fitness or objective value of each solution.

Selection operators in RGA

- Selection operators need the fitness or objective value of each solution.
- They decide which solutions get a chance to create offspring.

Selection operators in RGA

- Selection operators need the fitness or objective value of each solution.
- They decide which solutions get a chance to create offspring.
- Common choices include fitness proportionate selection, tournament selection, and ranking methods.

Selection operators in RGA

- Selection operators need the fitness or objective value of each solution.
- They decide which solutions get a chance to create offspring.
- Common choices include fitness proportionate selection, tournament selection, and ranking methods.
- In our RGA examples, we mostly use binary tournament selection.

Selection operators in RGA

- Selection operators need the fitness or objective value of each solution.
- They decide which solutions get a chance to create offspring.
- Common choices include fitness proportionate selection, tournament selection, and ranking methods.
- In our RGA examples, we mostly use binary tournament selection.

Because RGA solutions are real-valued, selection does not need to know the internal representation. It only compares solution quality.

Binary tournament selection: quick reminder

- 1 Randomly pick two candidate solutions.

Binary tournament selection: quick reminder

- ① Randomly pick two candidate solutions.
- ② Compare their fitness or objective values.

Binary tournament selection: quick reminder

- ➊ Randomly pick two candidate solutions.
- ➋ Compare their fitness or objective values.
- ➌ For minimization, the solution with smaller objective value wins.

Binary tournament selection: quick reminder

- ➊ Randomly pick two candidate solutions.
- ➋ Compare their fitness or objective values.
- ➌ For minimization, the solution with smaller objective value wins.
- ➍ Repeat until the mating pool is full.

Binary tournament selection: quick reminder

- ① Randomly pick two candidate solutions.
- ② Compare their fitness or objective values.
- ③ For minimization, the solution with smaller objective value wins.
- ④ Repeat until the mating pool is full.

Tournament selection is simple, does not require probability calculation, and works for both minimization and maximization when the comparison rule is clear.

Survival strategies: $(\mu + \lambda)$ and (μ, λ)

Suppose:

- μ parent solutions are available.

Survival strategies: $(\mu + \lambda)$ and (μ, λ)

Suppose:

- μ parent solutions are available.
- λ offspring solutions are created.

Survival strategies: $(\mu + \lambda)$ and (μ, λ)

Suppose:

- μ parent solutions are available.
- λ offspring solutions are created.

$(\mu + \lambda)$ **strategy**

- select next generation from parents + offspring

Survival strategies: $(\mu + \lambda)$ and (μ, λ)

Suppose:

- μ parent solutions are available.
- λ offspring solutions are created.

$(\mu + \lambda)$ **strategy**

- select next generation from parents + offspring
- good parents can survive directly

Survival strategies: $(\mu + \lambda)$ and (μ, λ)

Suppose:

- μ parent solutions are available.
- λ offspring solutions are created.

$(\mu + \lambda)$ **strategy**

- select next generation from parents + offspring
- good parents can survive directly
- select next generation from offspring only

Survival strategies: $(\mu + \lambda)$ and (μ, λ)

Suppose:

- μ parent solutions are available.
- λ offspring solutions are created.

$(\mu + \lambda)$ **strategy**

- select next generation from parents + offspring
- good parents can survive directly

(μ, λ) **strategy**

- select next generation from offspring only
- parents cannot survive directly

Simple survival example

Assume $\mu = 4$ and $\lambda = 4$ for a minimization problem.

Solution	Source	Objective J	Smaller is better
P1	parent	12	good
P2	parent	30	
P3	parent	18	
P4	parent	45	
O1	offspring	10	best
O2	offspring	26	
O3	offspring	15	
O4	offspring	50	

Simple survival example

Assume $\mu = 4$ and $\lambda = 4$ for a minimization problem.

Solution	Source	Objective J	Smaller is better
P1	parent	12	good
P2	parent	30	
P3	parent	18	
P4	parent	45	
O1	offspring	10	best
O2	offspring	26	
O3	offspring	15	
O4	offspring	50	

$(\mu + \lambda)$ keeps the best 4 from all 8: O1, P1, O3, P3.

(μ, λ) keeps the best 4 from offspring only: O1, O3, O2, O4.

Real-parameter crossover operators

- In RGA, crossover operators combine real-valued parent variables.

Real-parameter crossover operators

- In RGA, crossover operators combine real-valued parent variables.
- These operators are also called **blending** or **recombination** operators.

Real-parameter crossover operators

- In RGA, crossover operators combine real-valued parent variables.
- These operators are also called **blending** or **recombination** operators.
- They may generate offspring between parents or slightly outside the parent interval.

Real-parameter crossover operators

- In RGA, crossover operators combine real-valued parent variables.
- These operators are also called **blending** or **recombination** operators.
- They may generate offspring between parents or slightly outside the parent interval.

Common operators include:

- linear crossover

Real-parameter crossover operators

- In RGA, crossover operators combine real-valued parent variables.
- These operators are also called **blending** or **recombination** operators.
- They may generate offspring between parents or slightly outside the parent interval.

Common operators include:

- linear crossover
- blend crossover BLX- α

Real-parameter crossover operators

- In RGA, crossover operators combine real-valued parent variables.
- These operators are also called **blending** or **recombination** operators.
- They may generate offspring between parents or slightly outside the parent interval.

Common operators include:

- linear crossover
- blend crossover BLX- α
- simulated binary crossover (SBX)

Real-parameter crossover operators

- In RGA, crossover operators combine real-valued parent variables.
- These operators are also called **blending** or **recombination** operators.
- They may generate offspring between parents or slightly outside the parent interval.

Common operators include:

- linear crossover
- blend crossover BLX- α
- simulated binary crossover (SBX)
- fuzzy recombination (FR)

Real-parameter crossover operators

- In RGA, crossover operators combine real-valued parent variables.
- These operators are also called **blending** or **recombination** operators.
- They may generate offspring between parents or slightly outside the parent interval.

Common operators include:

- linear crossover
- blend crossover BLX- α
- simulated binary crossover (SBX)
- fuzzy recombination (FR)
- unimodal normally distributed crossover (UNDX)

Linear crossover operator

For one variable, assume two parents:

$$p_1 = x_i^{(1,t)}, \quad p_2 = x_i^{(2,t)}$$

- Linear crossover creates three possible offspring values.

Linear crossover operator

For one variable, assume two parents:

$$p_1 = x_i^{(1,t)}, \quad p_2 = x_i^{(2,t)}$$

- Linear crossover creates three possible offspring values.

$$o_1 = 0.5(p_1 + p_2)$$

Linear crossover operator

For one variable, assume two parents:

$$p_1 = x_i^{(1,t)}, \quad p_2 = x_i^{(2,t)}$$

- Linear crossover creates three possible offspring values.

$$o_1 = 0.5(p_1 + p_2)$$

$$o_2 = 1.5p_1 - 0.5p_2$$

Linear crossover operator

For one variable, assume two parents:

$$p_1 = x_i^{(1,t)}, \quad p_2 = x_i^{(2,t)}$$

- Linear crossover creates three possible offspring values.

$$o_1 = 0.5(p_1 + p_2)$$

$$o_2 = 1.5p_1 - 0.5p_2$$

$$o_3 = -0.5p_1 + 1.5p_2$$

Linear crossover operator

For one variable, assume two parents:

$$p_1 = x_i^{(1,t)}, \quad p_2 = x_i^{(2,t)}$$

- Linear crossover creates three possible offspring values.

$$o_1 = 0.5(p_1 + p_2)$$

$$o_2 = 1.5p_1 - 0.5p_2$$

$$o_3 = -0.5p_1 + 1.5p_2$$

After evaluating the three offspring, the two best are normally kept.

Linear crossover: simple numerical example

Suppose:

$$p_1 = 4, \quad p_2 = 10$$

- Middle offspring:

$$o_1 = 0.5(4 + 10) = 7$$

Linear crossover: simple numerical example

Suppose:

$$p_1 = 4, \quad p_2 = 10$$

- Middle offspring:

$$o_1 = 0.5(4 + 10) = 7$$

- Left-side extrapolated offspring:

$$o_2 = 1.5(4) - 0.5(10) = 1$$

Linear crossover: simple numerical example

Suppose:

$$p_1 = 4, \quad p_2 = 10$$

- Middle offspring:

$$o_1 = 0.5(4 + 10) = 7$$

- Left-side extrapolated offspring:

$$o_2 = 1.5(4) - 0.5(10) = 1$$

- Right-side extrapolated offspring:

$$o_3 = -0.5(4) + 1.5(10) = 13$$

Linear crossover: simple numerical example

Suppose:

$$p_1 = 4, \quad p_2 = 10$$

- Middle offspring:

$$o_1 = 0.5(4 + 10) = 7$$

- Left-side extrapolated offspring:

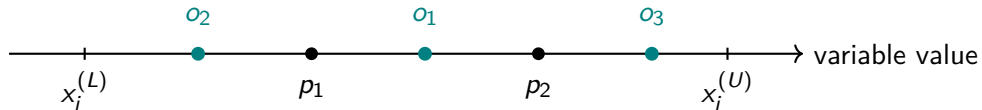
$$o_2 = 1.5(4) - 0.5(10) = 1$$

- Right-side extrapolated offspring:

$$o_3 = -0.5(4) + 1.5(10) = 13$$

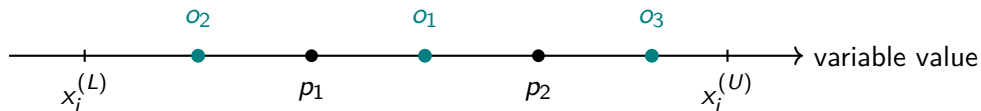
This operator can create one offspring between the parents and two offspring outside the parent interval.

Linear crossover: geometric view



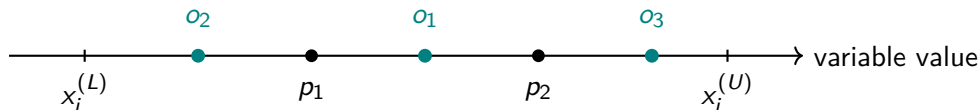
- o_1 is between the parents.

Linear crossover: geometric view



- o_1 is between the parents.
- o_2 and o_3 explore outside the parent interval.

Linear crossover: geometric view



- o_1 is between the parents.
- o_2 and o_3 explore outside the parent interval.
- Bounds must be checked if offspring go outside the allowed range.

- Assume two parent values for the i -th decision variable:

$$P_{1,i} < P_{2,i}$$

Blend crossover: BLX- α

- Assume two parent values for the i -th decision variable:

$$P_{1,i} < P_{2,i}$$

- BLX- α samples offspring from an expanded interval:

$$[P_{1,i} - \Delta, P_{2,i} + \Delta]$$

Blend crossover: BLX- α

- Assume two parent values for the i -th decision variable:

$$P_{1,i} < P_{2,i}$$

- BLX- α samples offspring from an expanded interval:

$$[P_{1,i} - \Delta, P_{2,i} + \Delta]$$

$$\Delta = \alpha(P_{2,i} - P_{1,i})$$

Blend crossover: BLX- α

- Assume two parent values for the i -th decision variable:

$$P_{1,i} < P_{2,i}$$

- BLX- α samples offspring from an expanded interval:

$$[P_{1,i} - \Delta, P_{2,i} + \Delta]$$

$$\Delta = \alpha(P_{2,i} - P_{1,i})$$

- Here, α controls how much the interval is expanded.

Blend crossover: BLX- α

- Assume two parent values for the i -th decision variable:

$$P_{1,i} < P_{2,i}$$

- BLX- α samples offspring from an expanded interval:

$$[P_{1,i} - \Delta, P_{2,i} + \Delta]$$

$$\Delta = \alpha(P_{2,i} - P_{1,i})$$

- Here, α controls how much the interval is expanded.
- If $\alpha = 0$, offspring are generated only between the parents.

Blend crossover: BLX- α

- Assume two parent values for the i -th decision variable:

$$P_{1,i} < P_{2,i}$$

- BLX- α samples offspring from an expanded interval:

$$[P_{1,i} - \Delta, P_{2,i} + \Delta]$$

$$\Delta = \alpha(P_{2,i} - P_{1,i})$$

- Here, α controls how much the interval is expanded.
- If $\alpha = 0$, offspring are generated only between the parents.
- If $\alpha > 0$, offspring may also be generated outside the parent interval.

BLX- α : generating offspring

- To generate two offspring, use two random numbers:

$$\gamma_i^{(1)} = (1 + 2\alpha)u_i^{(1)} - \alpha$$

$$\gamma_i^{(2)} = (1 + 2\alpha)u_i^{(2)} - \alpha$$

BLX- α : generating offspring

- To generate two offspring, use two random numbers:

$$\gamma_i^{(1)} = (1 + 2\alpha)u_i^{(1)} - \alpha$$

$$\gamma_i^{(2)} = (1 + 2\alpha)u_i^{(2)} - \alpha$$

- The offspring values are calculated as:

$$O_{1,i} = (1 - \gamma_i^{(1)})P_{1,i} + \gamma_i^{(1)}P_{2,i}$$

$$O_{2,i} = (1 - \gamma_i^{(2)})P_{1,i} + \gamma_i^{(2)}P_{2,i}$$

BLX- α : generating offspring

- To generate two offspring, use two random numbers:

$$\gamma_i^{(1)} = (1 + 2\alpha)u_i^{(1)} - \alpha$$

$$\gamma_i^{(2)} = (1 + 2\alpha)u_i^{(2)} - \alpha$$

- The offspring values are calculated as:

$$O_{1,i} = (1 - \gamma_i^{(1)})P_{1,i} + \gamma_i^{(1)}P_{2,i}$$

$$O_{2,i} = (1 - \gamma_i^{(2)})P_{1,i} + \gamma_i^{(2)}P_{2,i}$$

Notation

- P_1, P_2 : two parents
- O_1, O_2 : two offspring
- i : decision-variable number

BLX- α : generating offspring

- To generate two offspring, use two random numbers:

$$\gamma_i^{(1)} = (1 + 2\alpha)u_i^{(1)} - \alpha$$

$$\gamma_i^{(2)} = (1 + 2\alpha)u_i^{(2)} - \alpha$$

- The offspring values are calculated as:

$$O_{1,i} = (1 - \gamma_i^{(1)})P_{1,i} + \gamma_i^{(1)}P_{2,i}$$

$$O_{2,i} = (1 - \gamma_i^{(2)})P_{1,i} + \gamma_i^{(2)}P_{2,i}$$

Notation

- P_1, P_2 : two parents
- O_1, O_2 : two offspring
- i : decision-variable number

For example, if $X = [r, h]$, then $i = 1$ represents r , and $i = 2$ represents h .

Important clarification: when $\alpha = 0$

- If $\alpha = 0$, then $\Delta = 0$.

Important clarification: when $\alpha = 0$

- If $\alpha = 0$, then $\Delta = 0$.
- The sampling interval becomes:

$$[p_1, p_2]$$

Important clarification: when $\alpha = 0$

- If $\alpha = 0$, then $\Delta = 0$.
- The sampling interval becomes:

$$[p_1, p_2]$$

- This does **not** mean offspring must be exactly equal to the parents.

Important clarification: when $\alpha = 0$

- If $\alpha = 0$, then $\Delta = 0$.
- The sampling interval becomes:

$$[p_1, p_2]$$

- This does **not** mean offspring must be exactly equal to the parents.
- It means offspring are sampled only inside the parent interval.

Important clarification: when $\alpha = 0$

- If $\alpha = 0$, then $\Delta = 0$.
- The sampling interval becomes:

$$[p_1, p_2]$$

- This does **not** mean offspring must be exactly equal to the parents.
- It means offspring are sampled only inside the parent interval.

So, with $\alpha = 0$, offspring can be any value between p_1 and p_2 depending on the random number.

BLX- α : simple example

Suppose:

$$p_1 = 4, \quad p_2 = 10$$

- If $\alpha = 0$, then $\Delta = 0$ and the interval is $[4, 10]$.

BLX- α : simple example

Suppose:

$$p_1 = 4, \quad p_2 = 10$$

- If $\alpha = 0$, then $\Delta = 0$ and the interval is $[4, 10]$.
- If random numbers are $u_1 = 0.30$ and $u_2 = 0.70$, then:

$$o_1 = 4 + 0.30(10 - 4) = 5.8$$

$$o_2 = 4 + 0.70(10 - 4) = 8.2$$

BLX- α : simple example

Suppose:

$$p_1 = 4, \quad p_2 = 10$$

- If $\alpha = 0$, then $\Delta = 0$ and the interval is $[4, 10]$.
- If random numbers are $u_1 = 0.30$ and $u_2 = 0.70$, then:

$$o_1 = 4 + 0.30(10 - 4) = 5.8$$

$$o_2 = 4 + 0.70(10 - 4) = 8.2$$

- These offspring are inside the parent interval, not necessarily equal to the parents.

BLX- α : effect of α

Using the same parents:

$$p_1 = 4, \quad p_2 = 10$$

- If $\alpha = 0.5$:

$$\Delta = 0.5(10 - 4) = 3$$

BLX- α : effect of α

Using the same parents:

$$p_1 = 4, \quad p_2 = 10$$

- If $\alpha = 0.5$:

$$\Delta = 0.5(10 - 4) = 3$$

- The sampling interval becomes:

$$[4 - 3, 10 + 3] = [1, 13]$$

Using the same parents:

$$p_1 = 4, \quad p_2 = 10$$

- If $\alpha = 0.5$:

$$\Delta = 0.5(10 - 4) = 3$$

- The sampling interval becomes:

$$[4 - 3, 10 + 3] = [1, 13]$$

- Therefore, offspring may be generated outside the parent interval $[4, 10]$.

BLX- α : effect of α

Using the same parents:

$$p_1 = 4, \quad p_2 = 10$$

- If $\alpha = 0.5$:

$$\Delta = 0.5(10 - 4) = 3$$

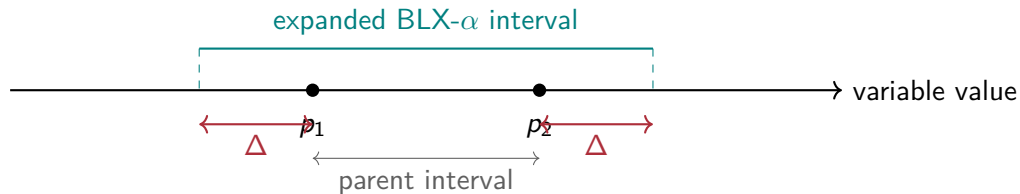
- The sampling interval becomes:

$$[4 - 3, 10 + 3] = [1, 13]$$

- Therefore, offspring may be generated outside the parent interval $[4, 10]$.

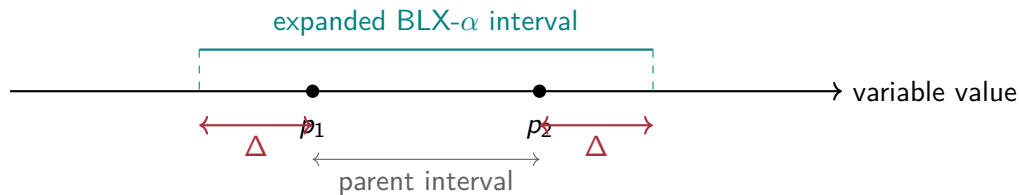
A larger α increases exploration because it gives a wider interval for offspring generation.

BLX- α : geometric view



- Offspring are sampled uniformly from the expanded interval.

BLX- α : geometric view



- Offspring are sampled uniformly from the expanded interval.
- The expansion amount depends on the distance between parents.

Why BLX- α is adaptive

- BLX- α uses the parent distance $(p_2 - p_1)$.

Why BLX- α is adaptive

- BLX- α uses the parent distance $(p_2 - p_1)$.
- If parents are far apart, the sampling interval becomes wide.

Why BLX- α is adaptive

- BLX- α uses the parent distance $(p_2 - p_1)$.
- If parents are far apart, the sampling interval becomes wide.
- If parents are close, the sampling interval becomes narrow.

Why BLX- α is adaptive

- BLX- α uses the parent distance $(p_2 - p_1)$.
- If parents are far apart, the sampling interval becomes wide.
- If parents are close, the sampling interval becomes narrow.
- Therefore, the search becomes naturally broad in early generations and focused in later generations.

Why BLX- α is adaptive

- BLX- α uses the parent distance ($p_2 - p_1$).
- If parents are far apart, the sampling interval becomes wide.
- If parents are close, the sampling interval becomes narrow.
- Therefore, the search becomes naturally broad in early generations and focused in later generations.

This is why BLX- α can be viewed as an adaptive real-valued recombination operator.

Simulated binary crossover (SBX)

- SBX was designed to mimic useful properties of binary single-point crossover in real-valued space.

Simulated binary crossover (SBX)

- SBX was designed to mimic useful properties of binary single-point crossover in real-valued space.
- It uses a spread factor β_i for each variable.

Simulated binary crossover (SBX)

- SBX was designed to mimic useful properties of binary single-point crossover in real-valued space.
- It uses a spread factor β_i for each variable.
- The spread factor controls how far the offspring are from each other relative to the parents.

Simulated binary crossover (SBX)

- SBX was designed to mimic useful properties of binary single-point crossover in real-valued space.
- It uses a spread factor β_i for each variable.
- The spread factor controls how far the offspring are from each other relative to the parents.

$$\frac{o_2 - o_1}{p_2 - p_1} = \beta_i$$

Simulated binary crossover (SBX)

- SBX was designed to mimic useful properties of binary single-point crossover in real-valued space.
- It uses a spread factor β_i for each variable.
- The spread factor controls how far the offspring are from each other relative to the parents.

$$\frac{o_2 - o_1}{p_2 - p_1} = \beta_i$$

If $\beta_i < 1$, offspring are closer together than parents. If $\beta_i > 1$, offspring are farther apart than parents.

SBX probability distribution

For SBX, the spread factor β_i follows this probability distribution:

$$P(\beta_i) = \begin{cases} 0.5(\eta_c + 1)\beta_i^{\eta_c}, & \beta_i \leq 1 \\ 0.5(\eta_c + 1)\frac{1}{\beta_i^{\eta_c+2}}, & \beta_i > 1 \end{cases}$$

- η_c is the crossover distribution index.

SBX probability distribution

For SBX, the spread factor β_i follows this probability distribution:

$$P(\beta_i) = \begin{cases} 0.5(\eta_c + 1)\beta_i^{\eta_c}, & \beta_i \leq 1 \\ 0.5(\eta_c + 1)\frac{1}{\beta_i^{\eta_c+2}}, & \beta_i > 1 \end{cases}$$

- η_c is the crossover distribution index.
- It controls how close the offspring are to the parent locations.

SBX probability distribution

For SBX, the spread factor β_i follows this probability distribution:

$$P(\beta_i) = \begin{cases} 0.5(\eta_c + 1)\beta_i^{\eta_c}, & \beta_i \leq 1 \\ 0.5(\eta_c + 1)\frac{1}{\beta_i^{\eta_c+2}}, & \beta_i > 1 \end{cases}$$

- η_c is the crossover distribution index.
- It controls how close the offspring are to the parent locations.
- Larger η_c gives stronger preference for near-parent offspring.

Calculating the spread factor in SBX

A random number $u_i \in [0, 1]$ is used to calculate β_i :

$$\beta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_c+1}}, & u_i \leq 0.5 \\ \left(\frac{1}{2(1-u_i)}\right)^{\frac{1}{\eta_c+1}}, & u_i > 0.5 \end{cases}$$

- If $u_i \leq 0.5$, usually $\beta_i \leq 1$.

Calculating the spread factor in SBX

A random number $u_i \in [0, 1]$ is used to calculate β_i :

$$\beta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_c+1}}, & u_i \leq 0.5 \\ \left(\frac{1}{2(1-u_i)}\right)^{\frac{1}{\eta_c+1}}, & u_i > 0.5 \end{cases}$$

- If $u_i \leq 0.5$, usually $\beta_i \leq 1$.
- If $u_i > 0.5$, usually $\beta_i > 1$.

Calculating the spread factor in SBX

A random number $u_i \in [0, 1]$ is used to calculate β_i :

$$\beta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_c+1}}, & u_i \leq 0.5 \\ \left(\frac{1}{2(1-u_i)}\right)^{\frac{1}{\eta_c+1}}, & u_i > 0.5 \end{cases}$$

- If $u_i \leq 0.5$, usually $\beta_i \leq 1$.
- If $u_i > 0.5$, usually $\beta_i > 1$.
- This allows both local search and expansion.

SBX: simple numerical example

Suppose for one variable:

$$p_1 = 4, \quad p_2 = 10, \quad \eta_c = 15$$

- Let $u = 0.30$. Then:

$$\beta = (2 \times 0.30)^{1/16} \approx 0.969$$

SBX: simple numerical example

Suppose for one variable:

$$p_1 = 4, \quad p_2 = 10, \quad \eta_c = 15$$

- Let $u = 0.30$. Then:

$$\beta = (2 \times 0.30)^{1/16} \approx 0.969$$

- The offspring become approximately:

$$o_1 \approx 4.09, \quad o_2 \approx 9.91$$

SBX: simple numerical example

Suppose for one variable:

$$p_1 = 4, \quad p_2 = 10, \quad \eta_c = 15$$

- Let $u = 0.30$. Then:

$$\beta = (2 \times 0.30)^{1/16} \approx 0.969$$

- The offspring become approximately:

$$o_1 \approx 4.09, \quad o_2 \approx 9.91$$

- Since $\beta < 1$, the offspring are slightly closer together than the parents.

Effect of η_c in SBX

- Large η_c gives higher probability of near-parent offspring.

Effect of η_c in SBX

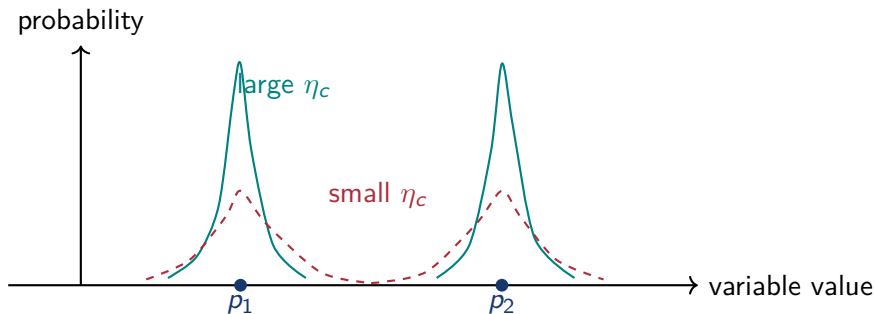
- Large η_c gives higher probability of near-parent offspring.
- Small η_c allows more distant offspring.

Effect of η_c in SBX

- Large η_c gives higher probability of near-parent offspring.
- Small η_c allows more distant offspring.
- Therefore, η_c controls exploitation versus exploration.

Effect of η_c in SBX

- Large η_c gives higher probability of near-parent offspring.
- Small η_c allows more distant offspring.
- Therefore, η_c controls exploitation versus exploration.



Interpreting the η_c graph

- The tall narrow peaks show that offspring are more likely to be created near p_1 and p_2 .

Interpreting the η_c graph

- The tall narrow peaks show that offspring are more likely to be created near p_1 and p_2 .
- This corresponds to a larger η_c .

Interpreting the η_c graph

- The tall narrow peaks show that offspring are more likely to be created near p_1 and p_2 .
- This corresponds to a larger η_c .
- The flatter wider curve allows offspring farther away from the parents.

Interpreting the η_c graph

- The tall narrow peaks show that offspring are more likely to be created near p_1 and p_2 .
- This corresponds to a larger η_c .
- The flatter wider curve allows offspring farther away from the parents.
- This corresponds to a smaller η_c .

Interpreting the η_c graph

- The tall narrow peaks show that offspring are more likely to be created near p_1 and p_2 .
- This corresponds to a larger η_c .
- The flatter wider curve allows offspring farther away from the parents.
- This corresponds to a smaller η_c .

The large η_c curve represents exploitation near the parents, while the small η_c curve represents more exploratory offspring.

Bounded SBX: why it is needed

The basic SBX idea may generate offspring outside the permitted variable range:

$$x_i^{(L)} \leq x_i \leq x_i^{(U)}$$

- In real engineering problems, variables have lower and upper limits.

Bounded SBX: why it is needed

The basic SBX idea may generate offspring outside the permitted variable range:

$$x_i^{(L)} \leq x_i \leq x_i^{(U)}$$

- In real engineering problems, variables have lower and upper limits.
- Therefore, we need a bounded version of SBX.

Bounded SBX: why it is needed

The basic SBX idea may generate offspring outside the permitted variable range:

$$x_i^{(L)} \leq x_i \leq x_i^{(U)}$$

- In real engineering problems, variables have lower and upper limits.
- Therefore, we need a bounded version of SBX.
- Bounded SBX modifies the sampling so that offspring stay inside the variable bounds.

Bounded SBX: why it is needed

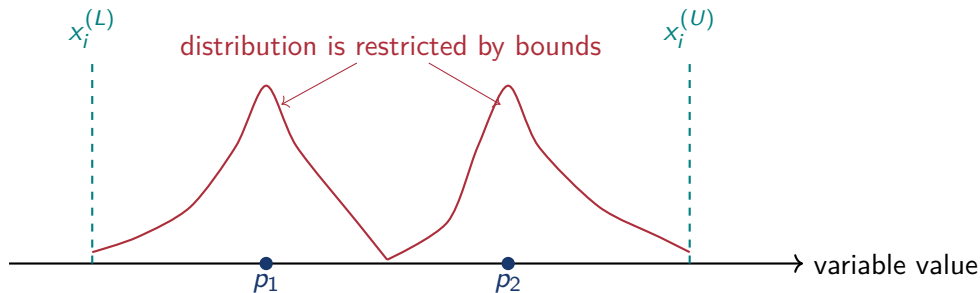
The basic SBX idea may generate offspring outside the permitted variable range:

$$x_i^{(L)} \leq x_i \leq x_i^{(U)}$$

- In real engineering problems, variables have lower and upper limits.
- Therefore, we need a bounded version of SBX.
- Bounded SBX modifies the sampling so that offspring stay inside the variable bounds.

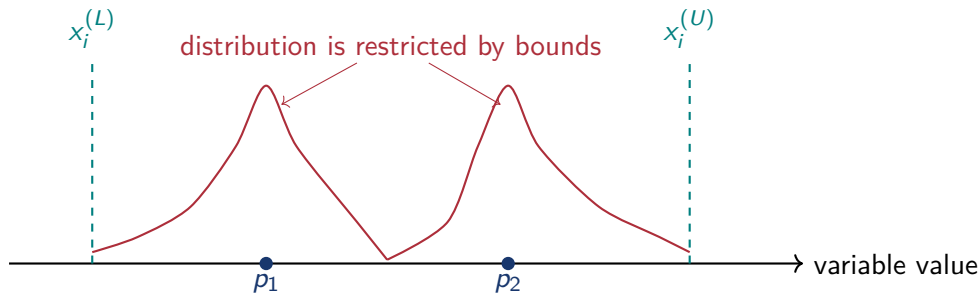
The distribution is restricted by $x_i^{(L)}$ and $x_i^{(U)}$ rather than allowing values over the whole real line.

Bounded SBX: graphical idea



- Values outside the dashed bounds are not accepted.

Bounded SBX: graphical idea



- Values outside the dashed bounds are not accepted.
- The bounded operator concentrates the valid probability inside the permitted range.

Important properties of SBX include:

- The difference between the offspring is proportional to the difference between parents.

$$o_2 - o_1 = \beta_i(p_2 - p_1)$$

Important properties of SBX include:

- The difference between the offspring is proportional to the difference between parents.

$$o_2 - o_1 = \beta_i(p_2 - p_1)$$

- Offspring near the parent solutions are more likely than distant offspring.

Important properties of SBX include:

- The difference between the offspring is proportional to the difference between parents.

$$o_2 - o_1 = \beta_i(p_2 - p_1)$$

- Offspring near the parent solutions are more likely than distant offspring.
- η_c controls how strongly the operator favors near-parent offspring.

SBX properties

Important properties of SBX include:

- The difference between the offspring is proportional to the difference between parents.

$$o_2 - o_1 = \beta_i(p_2 - p_1)$$

- Offspring near the parent solutions are more likely than distant offspring.
- η_c controls how strongly the operator favors near-parent offspring.

SBX is popular because it behaves like a real-valued version of binary crossover while still allowing controlled exploration.

Fuzzy recombination (FR) operator

- Fuzzy recombination is similar in spirit to SBX.

Fuzzy recombination (FR) operator

- Fuzzy recombination is similar in spirit to SBX.
- It uses a triangular probability distribution.

Fuzzy recombination (FR) operator

- Fuzzy recombination is similar in spirit to SBX.
- It uses a triangular probability distribution.
- The apex of the distribution is located at the parent solution.

Fuzzy recombination (FR) operator

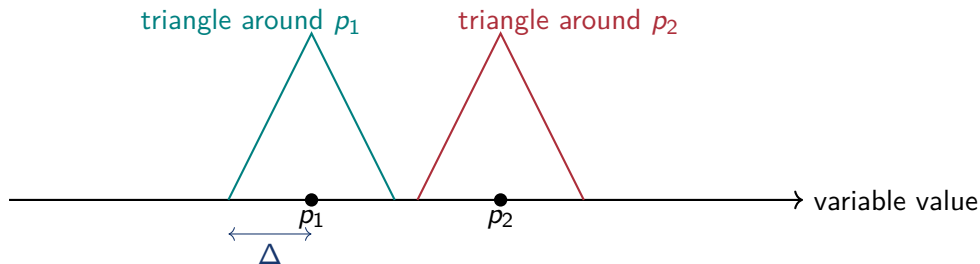
- Fuzzy recombination is similar in spirit to SBX.
- It uses a triangular probability distribution.
- The apex of the distribution is located at the parent solution.
- The base width is proportional to the difference between the two parents.

Fuzzy recombination (FR) operator

- Fuzzy recombination is similar in spirit to SBX.
- It uses a triangular probability distribution.
- The apex of the distribution is located at the parent solution.
- The base width is proportional to the difference between the two parents.

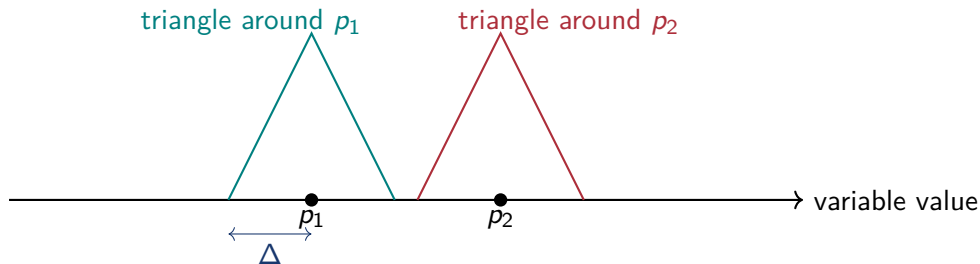
When $\eta_c = 1$, SBX and FR have a closely related behavior in terms of offspring distribution.

Fuzzy recombination: graphical idea



- The highest probability is near the parent.

Fuzzy recombination: graphical idea



- The highest probability is near the parent.
- Probability decreases as we move away from the parent.

Unimodal normally distributed crossover (UNDX)

- UNDX was proposed by Ono and Kobayashi.

Unimodal normally distributed crossover (UNDX)

- UNDX was proposed by Ono and Kobayashi.
- It uses three or more parent solutions.

Unimodal normally distributed crossover (UNDX)

- UNDX was proposed by Ono and Kobayashi.
- It uses three or more parent solutions.
- Two parents define the main search direction.

Unimodal normally distributed crossover (UNDX)

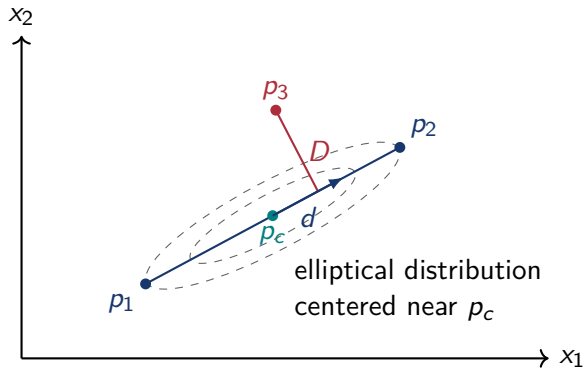
- UNDX was proposed by Ono and Kobayashi.
- It uses three or more parent solutions.
- Two parents define the main search direction.
- A third parent contributes information about the orthogonal direction.

Unimodal normally distributed crossover (UNDX)

- UNDX was proposed by Ono and Kobayashi.
- It uses three or more parent solutions.
- Two parents define the main search direction.
- A third parent contributes information about the orthogonal direction.

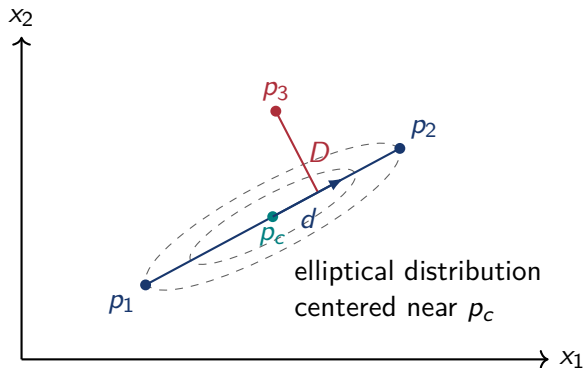
UNDX is useful for explaining that crossover in multiple dimensions can explore both along the parent line and perpendicular to it.

UNDX geometry



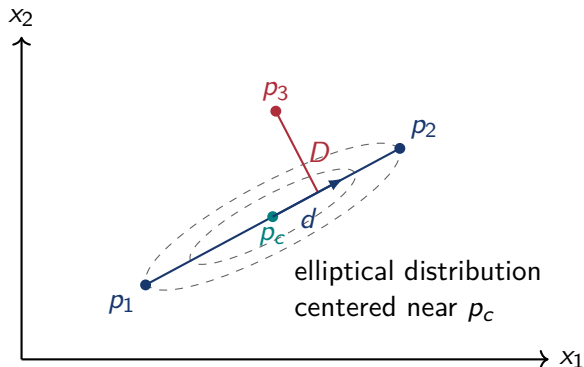
- $d = p_2 - p_1$ is the parallel direction.

UNDX geometry



- $d = p_2 - p_1$ is the parallel direction.
- $p_c = (p_1 + p_2)/2$ is the center.

UNDX geometry



- $d = p_2 - p_1$ is the parallel direction.
- $p_c = (p_1 + p_2)/2$ is the center.
- D is the perpendicular distance contributed by the third parent.

- The line from p_1 to p_2 gives the main direction of search.

- The line from p_1 to p_2 gives the main direction of search.
- The third parent p_3 tells how much spread is reasonable in the perpendicular direction.

- The line from p_1 to p_2 gives the main direction of search.
- The third parent p_3 tells how much spread is reasonable in the perpendicular direction.
- Offspring are generated from an ellipsoidal distribution.

- The line from p_1 to p_2 gives the main direction of search.
- The third parent p_3 tells how much spread is reasonable in the perpendicular direction.
- Offspring are generated from an ellipsoidal distribution.
- This allows search along a promising direction without losing diversity around that direction.

UNDX interpretation

- The line from p_1 to p_2 gives the main direction of search.
- The third parent p_3 tells how much spread is reasonable in the perpendicular direction.
- Offspring are generated from an ellipsoidal distribution.
- This allows search along a promising direction without losing diversity around that direction.

In simple words, UNDX says: move mainly along the line joining two parents, but also explore sideways using information from another parent.

Desired properties of crossover operators

A good crossover operator should satisfy two important ideas.

- **Population mean should not change unnecessarily.**

Desired properties of crossover operators

A good crossover operator should satisfy two important ideas.

- **Population mean should not change unnecessarily.**
 - Most crossover operators do not use fitness values.

Desired properties of crossover operators

A good crossover operator should satisfy two important ideas.

- **Population mean should not change unnecessarily.**
 - Most crossover operators do not use fitness values.
 - Therefore, crossover should not bias the search direction by itself.

Desired properties of crossover operators

A good crossover operator should satisfy two important ideas.

- **Population mean should not change unnecessarily.**
 - Most crossover operators do not use fitness values.
 - Therefore, crossover should not bias the search direction by itself.
- **Population diversity should not collapse too quickly.**

Desired properties of crossover operators

A good crossover operator should satisfy two important ideas.

- **Population mean should not change unnecessarily.**
 - Most crossover operators do not use fitness values.
 - Therefore, crossover should not bias the search direction by itself.
- **Population diversity should not collapse too quickly.**
 - Selection often reduces diversity.

Desired properties of crossover operators

A good crossover operator should satisfy two important ideas.

- **Population mean should not change unnecessarily.**
 - Most crossover operators do not use fitness values.
 - Therefore, crossover should not bias the search direction by itself.
- **Population diversity should not collapse too quickly.**
 - Selection often reduces diversity.
 - Crossover should help maintain or increase useful variance.

Why diversity matters

- If all solutions become too similar, the search may stop improving.

Why diversity matters

- If all solutions become too similar, the search may stop improving.
- This is called premature convergence.

Why diversity matters

- If all solutions become too similar, the search may stop improving.
- This is called premature convergence.
- A good crossover operator should create useful variety.

Why diversity matters

- If all solutions become too similar, the search may stop improving.
- This is called premature convergence.
- A good crossover operator should create useful variety.
- But too much variety may turn the search into almost random search.

Why diversity matters

- If all solutions become too similar, the search may stop improving.
- This is called premature convergence.
- A good crossover operator should create useful variety.
- But too much variety may turn the search into almost random search.

The practical goal is a balance: enough exploration to avoid premature convergence, and enough exploitation to improve good solutions.

Similarity among crossover operators

Different real-valued crossover operators may behave similarly for suitable parameter choices.

SBX η_c	BLX α	FR parameter d
2	0.662	1.095
3	0.500	0.707
5	0.419	0.433
10	0.380	0.226

Similarity among crossover operators

Different real-valued crossover operators may behave similarly for suitable parameter choices.

SBX η_c	BLX α	FR parameter d
2	0.662	1.095
3	0.500	0.707
5	0.419	0.433
10	0.380	0.226

This means that different operators can sometimes produce comparable diversity growth when their characteristic parameters are adjusted properly.

Mutation operators in RGA

In BGA, mutation flips bits. In RGA, mutation perturbs real numbers.

- Random mutation

Mutation operators in RGA

In BGA, mutation flips bits. In RGA, mutation perturbs real numbers.

- Random mutation
- Non-uniform mutation

Mutation operators in RGA

In BGA, mutation flips bits. In RGA, mutation perturbs real numbers.

- Random mutation
- Non-uniform mutation
- Normally distributed mutation

Mutation operators in RGA

In BGA, mutation flips bits. In RGA, mutation perturbs real numbers.

- Random mutation
- Non-uniform mutation
- Normally distributed mutation
- Polynomial mutation

Mutation operators in RGA

In BGA, mutation flips bits. In RGA, mutation perturbs real numbers.

- Random mutation
- Non-uniform mutation
- Normally distributed mutation
- Polynomial mutation

Mutation helps maintain diversity and allows local changes around current solutions.

Random mutation operator

- Random mutation may create a new value anywhere in the allowed range.

Random mutation operator

- Random mutation may create a new value anywhere in the allowed range.
- It is almost like randomly reinitializing that variable.

Random mutation operator

- Random mutation may create a new value anywhere in the allowed range.
- It is almost like randomly reinitializing that variable.
- It is independent of the parent value.

Random mutation operator

- Random mutation may create a new value anywhere in the allowed range.
- It is almost like randomly reinitializing that variable.
- It is independent of the parent value.

For a variable with bounds $x_i^{(L)}$ and $x_i^{(U)}$:

$$y_i = x_i^{(L)} + r_i \left(x_i^{(U)} - x_i^{(L)} \right), \quad r_i \in [0, 1]$$

Random mutation operator

- Random mutation may create a new value anywhere in the allowed range.
- It is almost like randomly reinitializing that variable.
- It is independent of the parent value.

For a variable with bounds $x_i^{(L)}$ and $x_i^{(U)}$:

$$y_i = x_i^{(L)} + r_i \left(x_i^{(U)} - x_i^{(L)} \right), \quad r_i \in [0, 1]$$

This mutation gives high exploration but may be too disruptive in later generations.

Random mutation: simple example

Suppose a variable has bounds:

$$0 \leq x \leq 10$$

- Current value is $x = 4.2$.

Random mutation: simple example

Suppose a variable has bounds:

$$0 \leq x \leq 10$$

- Current value is $x = 4.2$.
- Random number is $r = 0.73$.

Random mutation: simple example

Suppose a variable has bounds:

$$0 \leq x \leq 10$$

- Current value is $x = 4.2$.
- Random number is $r = 0.73$.
- New value becomes:

$$y = 0 + 0.73(10 - 0) = 7.3$$

Random mutation: simple example

Suppose a variable has bounds:

$$0 \leq x \leq 10$$

- Current value is $x = 4.2$.
- Random number is $r = 0.73$.
- New value becomes:

$$y = 0 + 0.73(10 - 0) = 7.3$$

- Notice that the new value does not depend on the old value 4.2.

Mutation in the vicinity of a parent

Sometimes we want mutation to stay near the parent.

- A user-defined maximum perturbation Δ_i can be used.

Mutation in the vicinity of a parent

Sometimes we want mutation to stay near the parent.

- A user-defined maximum perturbation Δ_i can be used.
- One simple form is:

$$y_i = x_i + (r_i - 0.5)\Delta_i$$

Mutation in the vicinity of a parent

Sometimes we want mutation to stay near the parent.

- A user-defined maximum perturbation Δ_i can be used.
- One simple form is:

$$y_i = x_i + (r_i - 0.5)\Delta_i$$

- Here, $r_i \in [0, 1]$.

Mutation in the vicinity of a parent

Sometimes we want mutation to stay near the parent.

- A user-defined maximum perturbation Δ_i can be used.
- One simple form is:

$$y_i = x_i + (r_i - 0.5)\Delta_i$$

- Here, $r_i \in [0, 1]$.
- The result must still satisfy the lower and upper bounds.

Mutation in the vicinity of a parent

Sometimes we want mutation to stay near the parent.

- A user-defined maximum perturbation Δ_i can be used.
- One simple form is:

$$y_i = x_i + (r_i - 0.5)\Delta_i$$

- Here, $r_i \in [0, 1]$.
- The result must still satisfy the lower and upper bounds.

This is more local than pure random mutation because it changes the value around the current parent.

Non-uniform mutation operator

- Non-uniform mutation changes its behavior as generations proceed.

Non-uniform mutation operator

- Non-uniform mutation changes its behavior as generations proceed.
- In early generations, larger changes are possible.

Non-uniform mutation operator

- Non-uniform mutation changes its behavior as generations proceed.
- In early generations, larger changes are possible.
- In later generations, smaller local changes become more likely.

Non-uniform mutation operator

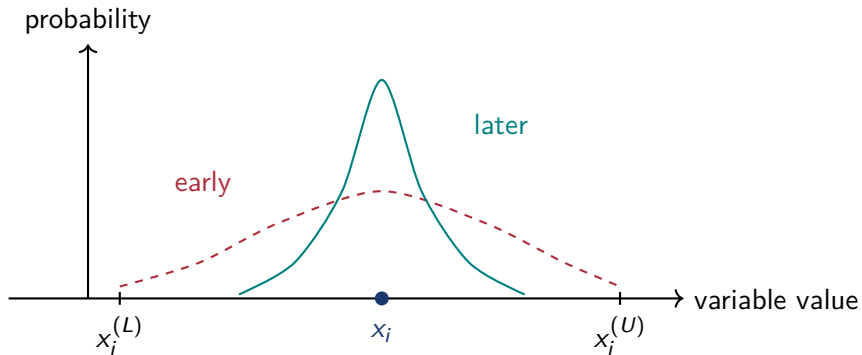
- Non-uniform mutation changes its behavior as generations proceed.
- In early generations, larger changes are possible.
- In later generations, smaller local changes become more likely.
- This supports exploration first and exploitation later.

Non-uniform mutation operator

- Non-uniform mutation changes its behavior as generations proceed.
- In early generations, larger changes are possible.
- In later generations, smaller local changes become more likely.
- This supports exploration first and exploitation later.

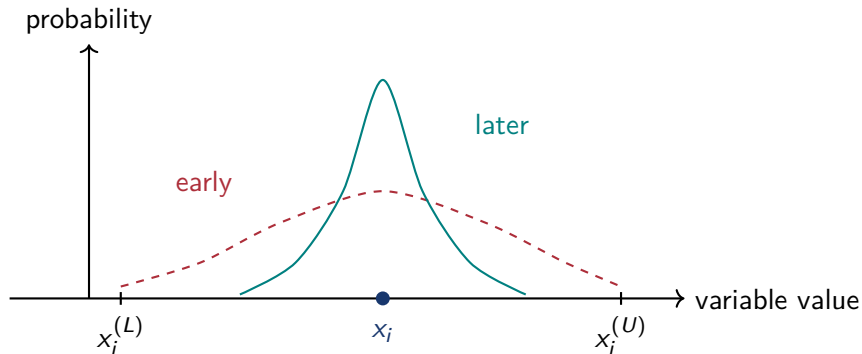
The main idea is simple: search broadly at the start, then fine-tune near the end.

Non-uniform mutation: graphical idea



- Early mutation can create farther points.

Non-uniform mutation: graphical idea



- Early mutation can create farther points.
- Later mutation is concentrated near the current value.

Normally distributed mutation

- A simple and popular method is Gaussian mutation.

Normally distributed mutation

- A simple and popular method is Gaussian mutation.
- It adds zero-mean normally distributed noise:

$$y_i = x_i + N(0, \sigma_i)$$

Normally distributed mutation

- A simple and popular method is Gaussian mutation.
- It adds zero-mean normally distributed noise:

$$y_i = x_i + N(0, \sigma_i)$$

- σ_i controls the mutation step size.

Normally distributed mutation

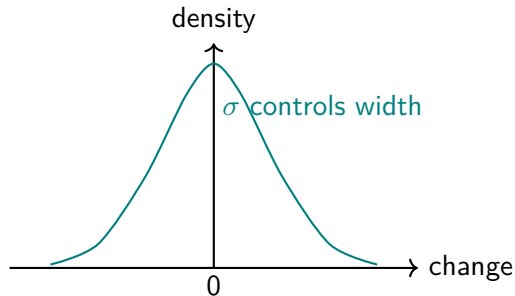
- A simple and popular method is Gaussian mutation.
- It adds zero-mean normally distributed noise:

$$y_i = x_i + N(0, \sigma_i)$$

- σ_i controls the mutation step size.
- Boundary limits must be respected after mutation.

Normally distributed mutation: graph and example

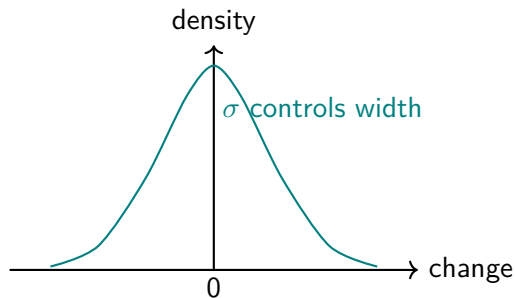
- Suppose $x = 4.2$ and $\sigma = 0.5$.



Normally distributed mutation: graph and example

- Suppose $x = 4.2$ and $\sigma = 0.5$.
- If the sampled normal noise is -0.6 , then:

$$y = 4.2 - 0.6 = 3.6$$

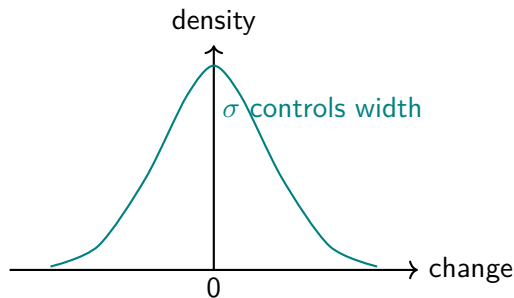


Normally distributed mutation: graph and example

- Suppose $x = 4.2$ and $\sigma = 0.5$.
- If the sampled normal noise is -0.6 , then:

$$y = 4.2 - 0.6 = 3.6$$

- Most changes are small, but larger changes are possible.



Polynomial mutation operator

- Polynomial mutation is commonly used with SBX in real-coded GA.

Polynomial mutation operator

- Polynomial mutation is commonly used with SBX in real-coded GA.
- It perturbs a variable using a polynomial probability distribution.

Polynomial mutation operator

- Polynomial mutation is commonly used with SBX in real-coded GA.
- It perturbs a variable using a polynomial probability distribution.
- The update form is:

$$y_i = x_i + (x_i^{(U)} - x_i^{(L)})\delta_i$$

Polynomial mutation operator

- Polynomial mutation is commonly used with SBX in real-coded GA.
- It perturbs a variable using a polynomial probability distribution.
- The update form is:

$$y_i = x_i + (x_i^{(U)} - x_i^{(L)})\delta_i$$

- δ_i controls the relative mutation step.

Polynomial mutation distribution

The mutation perturbation follows:

$$P(\delta) = 0.5(\eta_m + 1)(1 - |\delta|)^{\eta_m}, \quad -1 \leq \delta \leq 1$$

- η_m is the mutation distribution index.

Polynomial mutation distribution

The mutation perturbation follows:

$$P(\delta) = 0.5(\eta_m + 1)(1 - |\delta|)^{\eta_m}, \quad -1 \leq \delta \leq 1$$

- η_m is the mutation distribution index.
- Large η_m gives small local perturbations.

Polynomial mutation distribution

The mutation perturbation follows:

$$P(\delta) = 0.5(\eta_m + 1)(1 - |\delta|)^{\eta_m}, \quad -1 \leq \delta \leq 1$$

- η_m is the mutation distribution index.
- Large η_m gives small local perturbations.
- Small η_m allows larger jumps.

Polynomial mutation distribution

The mutation perturbation follows:

$$P(\delta) = 0.5(\eta_m + 1)(1 - |\delta|)^{\eta_m}, \quad -1 \leq \delta \leq 1$$

- η_m is the mutation distribution index.
- Large η_m gives small local perturbations.
- Small η_m allows larger jumps.

This is similar in role to η_c in SBX: it controls exploration versus exploitation.

Effect of η_m in polynomial mutation

- Large η_m : mutation is usually small and local.

Effect of η_m in polynomial mutation

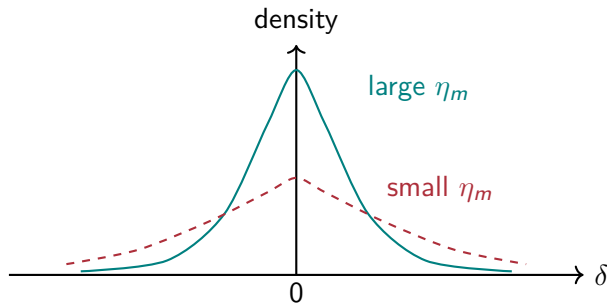
- Large η_m : mutation is usually small and local.
- Small η_m : mutation can produce larger jumps.

Effect of η_m in polynomial mutation

- Large η_m : mutation is usually small and local.
- Small η_m : mutation can produce larger jumps.
- Therefore, η_m also controls exploration versus exploitation.

Effect of η_m in polynomial mutation

- Large η_m : mutation is usually small and local.
- Small η_m : mutation can produce larger jumps.
- Therefore, η_m also controls exploration versus exploitation.



Polynomial mutation: simple example

Suppose:

$$0 \leq x \leq 10, \quad x = 4, \quad \eta_m = 20$$

- If random number $q = 0.30$, then $\delta \approx -0.024$.

Polynomial mutation: simple example

Suppose:

$$0 \leq x \leq 10, \quad x = 4, \quad \eta_m = 20$$

- If random number $q = 0.30$, then $\delta \approx -0.024$.
- New value becomes:

$$y = 4 + 10(-0.024) = 3.76$$

Polynomial mutation: simple example

Suppose:

$$0 \leq x \leq 10, \quad x = 4, \quad \eta_m = 20$$

- If random number $q = 0.30$, then $\delta \approx -0.024$.
- New value becomes:

$$y = 4 + 10(-0.024) = 3.76$$

- If $q = 0.80$, then $\delta \approx 0.043$.

Polynomial mutation: simple example

Suppose:

$$0 \leq x \leq 10, \quad x = 4, \quad \eta_m = 20$$

- If random number $q = 0.30$, then $\delta \approx -0.024$.
- New value becomes:

$$y = 4 + 10(-0.024) = 3.76$$

- If $q = 0.80$, then $\delta \approx 0.043$.
- New value becomes:

$$y = 4 + 10(0.043) = 4.43$$

Interpreting the polynomial mutation graph

- A large η_m gives a sharper peak near $\delta = 0$.

Interpreting the polynomial mutation graph

- A large η_m gives a sharper peak near $\delta = 0$.
- That means most mutations are small.

Interpreting the polynomial mutation graph

- A large η_m gives a sharper peak near $\delta = 0$.
- That means most mutations are small.
- A small η_m gives a wider and flatter curve.

Interpreting the polynomial mutation graph

- A large η_m gives a sharper peak near $\delta = 0$.
- That means most mutations are small.
- A small η_m gives a wider and flatter curve.
- That means larger jumps are more likely.

Interpreting the polynomial mutation graph

- A large η_m gives a sharper peak near $\delta = 0$.
- That means most mutations are small.
- A small η_m gives a wider and flatter curve.
- That means larger jumps are more likely.

Thus, high η_m supports local refinement, while small η_m supports wider exploration.

Boundary handling after mutation

After crossover or mutation, a variable may go outside its limits.

- Example: required range is $0 \leq x \leq 10$.

Boundary handling after mutation

After crossover or mutation, a variable may go outside its limits.

- Example: required range is $0 \leq x \leq 10$.
- If mutation produces $x = -1.2$, it is invalid.

Boundary handling after mutation

After crossover or mutation, a variable may go outside its limits.

- Example: required range is $0 \leq x \leq 10$.
- If mutation produces $x = -1.2$, it is invalid.
- A simple repair is clipping:

$$x = 0$$

Boundary handling after mutation

After crossover or mutation, a variable may go outside its limits.

- Example: required range is $0 \leq x \leq 10$.
- If mutation produces $x = -1.2$, it is invalid.
- A simple repair is clipping:

$$x = 0$$

- Another option is resampling or reflecting the value.

Boundary handling after mutation

After crossover or mutation, a variable may go outside its limits.

- Example: required range is $0 \leq x \leq 10$.
- If mutation produces $x = -1.2$, it is invalid.
- A simple repair is clipping:

$$x = 0$$

- Another option is resampling or reflecting the value.

Clipping is easy but in research implementations, the choice of repair method can also work better.

To test RGA operators, benchmark functions are commonly used.

- Rosenbrock function

To test RGA operators, benchmark functions are commonly used.

- Rosenbrock function
- Himmelblau function

To test RGA operators, benchmark functions are commonly used.

- Rosenbrock function
- Himmelblau function
- Rastrigin function

To test RGA operators, benchmark functions are commonly used.

- Rosenbrock function
- Himmelblau function
- Rastrigin function
- Ackley function

RGA benchmark testing

To test RGA operators, benchmark functions are commonly used.

- Rosenbrock function
- Himmelblau function
- Rastrigin function
- Ackley function

Benchmark functions help us compare operators under known mathematical landscapes before applying them to engineering design problems.

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$
- population size: $N = 40$

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$
- population size: $N = 40$
- number of generations: $T = 200$

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$
- population size: $N = 40$
- number of generations: $T = 200$
- crossover probability: $p_c = 1.0$

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$
- population size: $N = 40$
- number of generations: $T = 200$
- crossover probability: $p_c = 1.0$
- mutation probability: $p_m = 1/n = 0.5$

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$
- population size: $N = 40$
- number of generations: $T = 200$
- crossover probability: $p_c = 1.0$
- mutation probability: $p_m = 1/n = 0.5$
- binary tournament selection

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$
- population size: $N = 40$
- number of generations: $T = 200$
- crossover probability: $p_c = 1.0$
- mutation probability: $p_m = 1/n = 0.5$
- binary tournament selection
- SBX crossover with $\eta_c = 15$

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$
- population size: $N = 40$
- number of generations: $T = 200$
- crossover probability: $p_c = 1.0$
- mutation probability: $p_m = 1/n = 0.5$
- binary tournament selection
- SBX crossover with $\eta_c = 15$
- polynomial mutation with $\eta_m = 20$

Rosenbrock function settings

For the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Typical RGA settings may be:

- number of variables: $n = 2$
- population size: $N = 40$
- number of generations: $T = 200$
- crossover probability: $p_c = 1.0$
- mutation probability: $p_m = 1/n = 0.5$
- binary tournament selection
- SBX crossover with $\eta_c = 15$
- polynomial mutation with $\eta_m = 20$
- $(\mu + \lambda)$ survival strategy

Fair comparison of operators

When comparing different operators, use the same conditions.

- Same objective function

Fair comparison of operators

When comparing different operators, use the same conditions.

- Same objective function
- Same population size

Fair comparison of operators

When comparing different operators, use the same conditions.

- Same objective function
- Same population size
- Same number of generations

Fair comparison of operators

When comparing different operators, use the same conditions.

- Same objective function
- Same population size
- Same number of generations
- Same stopping criterion

Fair comparison of operators

When comparing different operators, use the same conditions.

- Same objective function
- Same population size
- Same number of generations
- Same stopping criterion
- Multiple independent runs

Fair comparison of operators

When comparing different operators, use the same conditions.

- Same objective function
- Same population size
- Same number of generations
- Same stopping criterion
- Multiple independent runs
- Compare best, average, and standard deviation of final results

Fair comparison of operators

When comparing different operators, use the same conditions.

- Same objective function
- Same population size
- Same number of generations
- Same stopping criterion
- Multiple independent runs
- Compare best, average, and standard deviation of final results

Because RGA is stochastic, one run is not enough for a fair comparison.

Quick comparison of operators

Operator	Main idea
Linear crossover	creates fixed combinations of parents
BLX- α	samples uniformly from an expanded parent interval
SBX	uses a spread factor distribution inspired by binary crossover
FR	uses triangular distributions around parents
UNDX	uses multiple parents and an ellipsoidal distribution
Random mutation	reinitializes a variable randomly
Normal mutation	adds Gaussian noise
Polynomial mutation	uses polynomial perturbation controlled by η_m

Class discussion questions

- Why does selection not change much from BGA to RGA?

Class discussion questions

- Why does selection not change much from BGA to RGA?
- What is the difference between $(\mu + \lambda)$ and (μ, λ) survival?

Class discussion questions

- Why does selection not change much from BGA to RGA?
- What is the difference between $(\mu + \lambda)$ and (μ, λ) survival?
- Why can BLX- α generate offspring outside the parent interval?

Class discussion questions

- Why does selection not change much from BGA to RGA?
- What is the difference between $(\mu + \lambda)$ and (μ, λ) survival?
- Why can BLX- α generate offspring outside the parent interval?
- What happens when η_c is increased in SBX?

Class discussion questions

- Why does selection not change much from BGA to RGA?
- What is the difference between $(\mu + \lambda)$ and (μ, λ) survival?
- Why can BLX- α generate offspring outside the parent interval?
- What happens when η_c is increased in SBX?
- Why does polynomial mutation need boundary handling?

Class discussion questions

- Why does selection not change much from BGA to RGA?
- What is the difference between $(\mu + \lambda)$ and (μ, λ) survival?
- Why can BLX- α generate offspring outside the parent interval?
- What happens when η_c is increased in SBX?
- Why does polynomial mutation need boundary handling?
- Which mutation operator is more exploratory: random mutation or polynomial mutation with large η_m ?

Quick quiz

1. In **BLX- α** , increasing α mainly:

- A. reduces the search interval

Quick quiz

1. In **BLX- α** , increasing α mainly:
 - A. reduces the search interval
 - B. expands the possible offspring interval

1. In **BLX- α** , increasing α mainly:
 - A. reduces the search interval
 - B. expands the possible offspring interval
 - C. removes mutation

1. In BLX- α , increasing α mainly:

- A. reduces the search interval
- B. expands the possible offspring interval
- C. removes mutation
- D. changes minimization to maximization

Quick quiz

1. In BLX- α , increasing α mainly:

- A. reduces the search interval
- B. expands the possible offspring interval
- C. removes mutation
- D. changes minimization to maximization

Answer: B

- A. offspring closer to parents

Quick quiz

1. In BLX- α , increasing α mainly:

- A. reduces the search interval
- B. expands the possible offspring interval
- C. removes mutation
- D. changes minimization to maximization

Answer: B

2. In SBX, a larger η_c generally creates:

- A. offspring closer to parents
- B. completely random offspring

Quick quiz

1. In BLX- α , increasing α mainly:

- A. reduces the search interval
- B. expands the possible offspring interval
- C. removes mutation
- D. changes minimization to maximization

Answer: B

2. In SBX, a larger η_c generally creates:

- A. offspring closer to parents
- B. completely random offspring
- C. only invalid offspring

Quick quiz

1. In **BLX- α** , increasing α mainly:

- A. reduces the search interval
- B. expands the possible offspring interval
- C. removes mutation
- D. changes minimization to maximization

Answer: B

2. In **SBX**, a larger η_c generally creates:

- A. offspring closer to parents
- B. completely random offspring
- C. only invalid offspring
- D. no offspring

Quick quiz

1. In **BLX- α** , increasing α mainly:

- A. reduces the search interval
- B. expands the possible offspring interval
- C. removes mutation
- D. changes minimization to maximization

Answer: B

2. In **SBX**, a larger η_c generally creates:

- A. offspring closer to parents
- B. completely random offspring
- C. only invalid offspring
- D. no offspring

Quick quiz

1. In **BLX- α** , increasing α mainly:

- A. reduces the search interval
- B. expands the possible offspring interval
- C. removes mutation
- D. changes minimization to maximization

Answer: B

2. In **SBX**, a larger η_c generally creates:

- A. offspring closer to parents
- B. completely random offspring
- C. only invalid offspring
- D. no offspring

Quick quiz

1. In BLX- α , increasing α mainly:

- A. reduces the search interval
- B. expands the possible offspring interval
- C. removes mutation
- D. changes minimization to maximization

Answer: B

2. In SBX, a larger η_c generally creates:

- A. offspring closer to parents
- B. completely random offspring
- C. only invalid offspring
- D. no offspring

Answer: A

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips
- D. no change in any variable

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips
- D. no change in any variable

Answer: B

- A. binary strings only

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips
- D. no change in any variable

Answer: B

4. UNDX mainly uses:

- A. binary strings only
- B. three or more parents and an ellipsoidal distribution

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips
- D. no change in any variable

Answer: B

4. UNDX mainly uses:

- A. binary strings only
- B. three or more parents and an ellipsoidal distribution
- C. mutation only

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips
- D. no change in any variable

Answer: B

4. UNDX mainly uses:

- A. binary strings only
- B. three or more parents and an ellipsoidal distribution
- C. mutation only
- D. no population

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips
- D. no change in any variable

Answer: B

4. UNDX mainly uses:

- A. binary strings only
- B. three or more parents and an ellipsoidal distribution
- C. mutation only
- D. no population

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips
- D. no change in any variable

Answer: B

4. UNDX mainly uses:

- A. binary strings only
- B. three or more parents and an ellipsoidal distribution
- C. mutation only
- D. no population

3. Polynomial mutation with large η_m usually produces:

- A. large random jumps only
- B. small local changes
- C. binary bit flips
- D. no change in any variable

Answer: B

4. UNDX mainly uses:

- A. binary strings only
- B. three or more parents and an ellipsoidal distribution
- C. mutation only
- D. no population

Answer: B

End-of-lecture summary

In this lecture, we discussed:

- selection and survivor operators in RGA

End-of-lecture summary

In this lecture, we discussed:

- selection and survivor operators in RGA
- linear crossover, BLX- α , SBX, FR, and UNDX

End-of-lecture summary

In this lecture, we discussed:

- selection and survivor operators in RGA
- linear crossover, BLX- α , SBX, FR, and UNDX
- why η_c controls the behavior of SBX

End-of-lecture summary

In this lecture, we discussed:

- selection and survivor operators in RGA
- linear crossover, BLX- α , SBX, FR, and UNDX
- why η_c controls the behavior of SBX
- desired properties of crossover operators

End-of-lecture summary

In this lecture, we discussed:

- selection and survivor operators in RGA
- linear crossover, BLX- α , SBX, FR, and UNDX
- why η_c controls the behavior of SBX
- desired properties of crossover operators
- random, non-uniform, normal, and polynomial mutation

End-of-lecture summary

In this lecture, we discussed:

- selection and survivor operators in RGA
- linear crossover, BLX- α , SBX, FR, and UNDX
- why η_c controls the behavior of SBX
- desired properties of crossover operators
- random, non-uniform, normal, and polynomial mutation
- why η_m controls the size of polynomial mutation steps

End-of-lecture summary

In this lecture, we discussed:

- selection and survivor operators in RGA
- linear crossover, BLX- α , SBX, FR, and UNDX
- why η_c controls the behavior of SBX
- desired properties of crossover operators
- random, non-uniform, normal, and polynomial mutation
- why η_m controls the size of polynomial mutation steps
- how benchmark functions are used to test RGA